

Fixed-Window Draft-Model Speculative Decoding versus EAGLE-3: A Like-for-Like, Energy-Measured Comparison in vLLM and SGLang

Zanno Jacklin

Independent Researcher, Creepybits

business@zanno.se

Preprint, October 2026 • DOI: [10.5281/zenodo.22210487](https://doi.org/10.5281/zenodo.22210487)

Abstract—Speculative decoding with a separate small draft model is a well-established technique, shipped today in vLLM, SGLang and Hugging Face Transformers. Newer methods such as EAGLE-3 replace the draft model with a lightweight draft head trained for one specific target model. We compare a fixed-window draft-model configuration (“K5”: Llama-3.2-1B drafting $K=5$ tokens for Llama-3.1-8B) against EAGLE-3, EAGLE-1 and n-gram drafting, measuring throughput and GPU energy per token. The comparison is like-for-like: each method runs inside the same engine as its competitors, each method is given several configurations, the best configuration is chosen on half of 27 prompts and reported on the other half, and every run executes in a fresh process bracketed by baseline runs. At batch size 1 on an RTX 5090, K5 increases throughput by +79% (SGLang) and +60% (vLLM) and reduces energy per token by 55% and 53% relative to non-speculative decoding. The best EAGLE-3 configuration is faster in both engines, by 14% and 8% respectively; on energy per token the result is split (K5 uses 10% more in SGLang and 9% less in vLLM). K5 thus attains roughly three quarters to four fifths of EAGLE-3’s throughput gain with comparable energy savings, without requiring a target-specific trained draft head. We also report that every speculative method tested diverges from its own engine’s non-speculative output on most prompts in bf16, and that a controlled, cache-free reimplementation reproduces the mechanism’s speed and energy scaling with acceptance rate at 100% token fidelity.

Index Terms—speculative decoding, EAGLE-3, energy-efficient inference, large language models, GPU power measurement, benchmarking methodology

I. INTRODUCTION

Autoregressive decoding produces one token per full forward pass of the target model, leaving much of a modern GPU’s parallel capacity unused at small batch sizes. Speculative decoding [1], [2] lets a cheaper drafter propose several tokens that the target model then verifies in a single forward pass, keeping the longest prefix that matches the target’s own choice. With greedy decoding the accepted output is, in exact arithmetic, identical to what the target alone would produce.

The original formulation uses a separate small *draft model* from the same family. This is not a new technique: it is available as `draft_model` in vLLM [3], as `STANDALONE` in SGLang [4], and as assisted generation in Hugging Face

Transformers [5]. Later methods replace the draft model with a lightweight head trained on the target model’s internal features, most recently EAGLE-3 [6], [7], which is widely regarded as the stronger approach.

This paper asks a narrow practical question: *inside production serving engines, how does a simple fixed-window draft-model configuration compare with EAGLE-3 on both speed and energy?* Our contribution is measurement, not a new algorithm: (i) a like-for-like benchmarking protocol designed so that no method is handicapped by configuration choice, run order, cross-request memory or engine differences (Section II); (ii) results in two engines, including GPU energy per token (Section III); and (iii) a controlled, cache-free reimplementation that isolates the mechanism’s behaviour (Section IV).

A. Related work

Beyond draft models and EAGLE, adaptive methods vary the draft length at run time; AdaEDL [8], for example, stops drafting early using an entropy-based bound on acceptance probability and reports 10–57% improvement over static draft lengths. This paper keeps the draft length fixed. An entropy-gated adaptive variant was explored separately in the parent project and has not, so far, outperformed the fixed window in single-request testing; we make no claims about it here.

II. SETUP AND LIKE-FOR-LIKE PROTOCOL

A. Models, hardware and software

Target: Llama-3.1-8B-Instruct. Draft model: Llama-3.2-1B-Instruct. EAGLE checkpoints: the public EAGLE-1 and EAGLE-3 heads for Llama-3.1-8B-Instruct. All models in bfloat16, greedy decoding, batch size 1, up to 250 generated tokens per request with natural end-of-sequence, prompts formatted with the target’s chat template. One NVIDIA RTX 5090 (Blackwell) under WSL2, without clock locking. Engines: vLLM 0.27.1 and SGLang 0.5.20, each in its own Python environment (PyTorch 2.13).

B. Energy measurement

GPU energy is read from the on-board hardware energy counter (`nvmlDeviceGetTotalEnergyConsumption`) at the start and end of each timed request and divided by the number of tokens produced. Power and temperature are sampled at 100 Hz for warmup and drift diagnostics.

C. Protocol

Same engine only. Methods are compared only within one engine; numbers from vLLM and SGLang are never placed against each other, since they differ by engineering rather than algorithm.

Several configurations per method. vLLM: draft model with $K \in \{3, 5, 7\}$, EAGLE-3 with $K \in \{2, 3, 5\}$, EAGLE-1 with $K \in \{3, 5\}$, n-gram with $K=5$. SGLang: draft model with $K \in \{3, 5, 7\}$ (chain), EAGLE-3 with $K \in \{3, 5\}$ (chain) and one tree configuration (5 steps, top- k 8, 32 draft tokens), EAGLE-1 with $K=5$, n-gram with 16 draft tokens. K5 is the draft model at $K=5$.

Tuning separated from evaluation. 27 prompts are used: the project’s three reference prompts (a poem, a physics explanation, a code task) plus 24 original prompts, three in each of the eight categories used by MT-Bench [9] (not the MT-Bench questions themselves). Each method’s best configuration is selected on 14 prompts and reported on the remaining 13 (alternating within category), so methods with more configurations do not gain from selecting and grading on the same data.

No repeated exposure. Each configuration runs in a fresh engine process, and each timed prompt is generated exactly once per process. Repetition (two per configuration) happens across processes. This prevents methods that keep memory across requests from drafting their own previous answer to an identical greedy prompt. Warmup uses separate prompts that are never timed, and prefix caching is disabled in both engines.

Drift control. Each engine block starts and ends with a baseline run (no speculation). Speedups are computed per prompt against the mean of the two baselines of the same block. Configuration order is randomized per repetition (seed recorded) and engine order alternates. Each run first warms the GPU in a closed loop until per-prompt power and temperature stop drifting (minimum 150 s), and a short untimed generation on a warmup prompt precedes every timed request.

Statistics. Per-prompt speedups are combined with a geometric mean; 95% ranges are bootstrap intervals over prompts (2000 resamples).

Sanity checks. A configuration is shown but excluded from “best” if its mean accept length (tokens per verification step) is below 1.05, if its speedup exceeds its accept length (impossible, since each verification step costs at least one ordinary decoding step), or if output lengths differ from baseline on more than 20% of prompts.

Known engine asymmetry. vLLM 0.27.1 runs `draft_model` on its older V1 model runner with asynchronous scheduling disabled, while EAGLE-3 runs on the V2 model runner with asynchronous scheduling enabled

TABLE I
K5 RELATIVE TO THE BEST CONFIGURATION OF EACH OTHER METHOD (EVALUATION PROMPTS, 13). SPEED: POSITIVE MEANS K5 IS FASTER. ENERGY: NEGATIVE MEANS K5 USES LESS ENERGY PER TOKEN.

Engine	Opponent (best config)	K5 speed [95%]	K5 faster	K5 energy
SGLang	EAGLE-3 (tree)	−13.6% [−19.0, −8.5]	0/13	+10.2%
SGLang	N-gram	+65.9% [+57.1, +75.2]	13/13	−41.5%
vLLM	EAGLE-3 ($K=5$)	−8.2% [−14.5, −1.5]	4/13	−9.1%
vLLM	EAGLE-3, async off ($K=3$)	−12.7% [−17.3, −7.2]	2/13	−5.5%
vLLM	EAGLE-1 ($K=3$)	−1.0% [−5.3, +3.5]	7/13	−18.2%
vLLM	N-gram	+70.9% [+62.5, +79.7]	13/13	−45.0%

(both confirmed from vLLM’s own logs). We additionally ran EAGLE-3 with asynchronous scheduling disabled (“EAGLE-3, async off”); the model-runner difference could not be matched.

III. RESULTS

A. K5 versus each method’s best configuration

Figure 1 and Table I summarise the comparison on the evaluation prompts. All 46 runs completed; baseline draft within a block (closing versus opening baseline) was 1.5% in SGLang and 4.1% in vLLM, the latter above our 3% warning threshold, so small vLLM differences should be read with caution.

Speed. The best EAGLE-3 configuration is faster than K5 in both engines: by 13.6% in SGLang, where the tree configuration wins, and by 8.2% in vLLM. In SGLang, EAGLE-3 in chain mode ($K=3$ or 5) is *slower* than K5 (Table II); only the tree configuration overtakes it. Measured as a share of the best EAGLE-3’s throughput gain over no speculation, K5 reaches about 74% in SGLang (+79.4% vs. +107.5%) and 81% in vLLM (+59.9% vs. +74.2%). By prompt category (all prompts), EAGLE-3’s advantage is smallest on the math, coding and roleplay prompts and largest on the humanities prompts and the three reference prompts; in vLLM, K5 is ahead on the math category (+75.5% vs. +67.1%).

Energy. The energy result is split: K5 uses 10.2% more energy per token than the best EAGLE-3 in SGLang and 9.1% less in vLLM. In both engines K5 roughly halves energy per token relative to non-speculative decoding (−55.2% SGLang, −53.4% vLLM).

N-gram and EAGLE-1. K5 is far faster than n-gram drafting in both engines, and is statistically indistinguishable from EAGLE-1 in vLLM on speed while using 18% less energy per token.

B. Choice of K

The fixed window $K=5$ is as good as any window tested for the draft model. In SGLang it is the best draft configuration; in vLLM the tuning half selected $K=3$, but on the evaluation prompts K5 and $K=3$ are indistinguishable (K5 +1.0%, 95% range −2.8% to +4.8%).

C. Other observations

Broken configurations are caught. SGLang’s EAGLE-1 had an accept length of 1.01 (drafts almost never accepted)

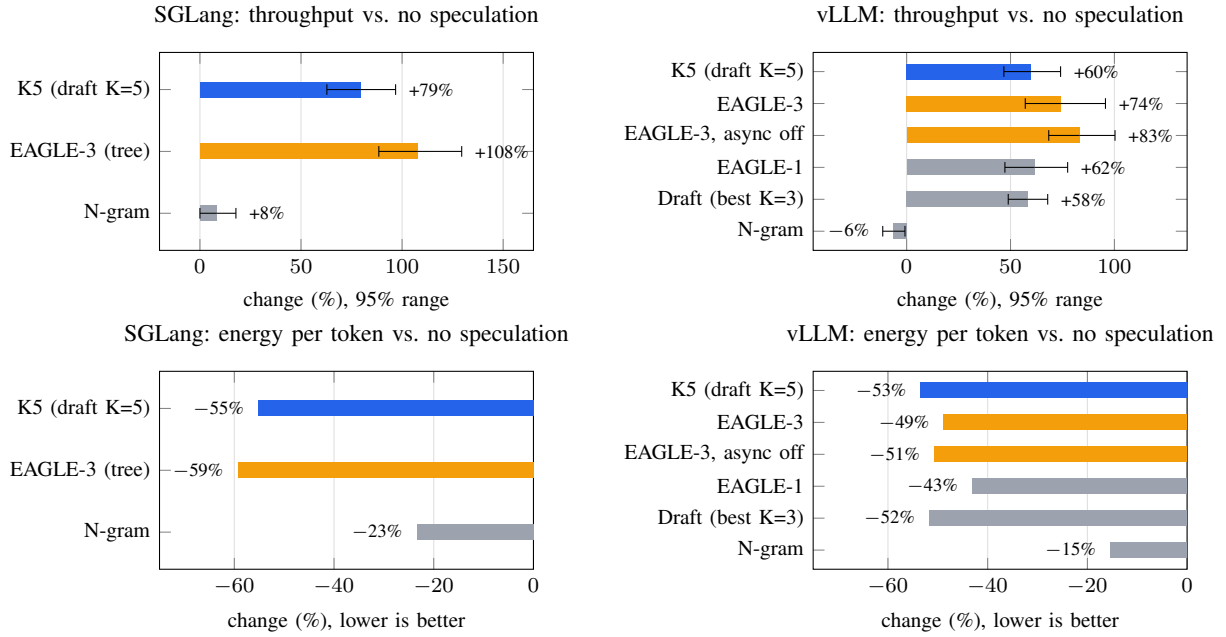


Fig. 1. Like-for-like comparison: each method at its best configuration (chosen on 14 prompts), reported on the 13 evaluation prompts. Batch size 1, RTX 5090. Error bars: 95% bootstrap range over prompts. Engines are shown separately and are not comparable with each other. SGLang’s EAGLE-1 is omitted (it failed the sanity check; Section III-C).

TABLE II
EVERY CONFIGURATION, ALL 27 PROMPTS, TWO RUNS EACH. ACCEPT LENGTH IS THE MEAN NUMBER OF TOKENS PRODUCED PER TARGET VERIFICATION STEP. FIDELITY: MEAN SHARE OF EACH OUTPUT THAT MATCHES THE SAME ENGINE’S FIRST BASELINE RUN BEFORE THE FIRST DIFFERING TOKEN / SHARE OF OUTPUTS THAT ARE IDENTICAL. BASELINE RUN VERSUS BASELINE RUN: 100% / 100% IN BOTH ENGINES.

SGLang 0.5.20					vLLM 0.27.1				
Config	Speed [95%]	Energy	Acc. len	Fidelity	Config	Speed [95%]	Energy	Acc. len	Fidelity
EAGLE-3 tree	+106.8 [+94.7, +119.7]	-59.2	3.53	62 / 41	EAGLE-3 K3 async off	+79.6 [+70.4, +90.2]	-50.1	2.21	65 / 44
K5	+76.7 [+66.1, +88.3]	-54.3	4.28	52 / 30	EAGLE-3 K5 async off	+76.1 [+64.0, +89.1]	-50.0	2.47	72 / 48
Draft K7	+74.0 [+60.4, +89.7]	-52.9	5.07	55 / 30	EAGLE-3 K3	+74.0 [+64.9, +84.2]	-49.4	2.21	65 / 44
Draft K3	+65.8 [+58.6, +73.2]	-51.5	3.20	58 / 37	EAGLE-3 K5	+73.8 [+62.2, +86.1]	-48.9	2.47	72 / 48
EAGLE-3 K5	+55.2 [+45.1, +66.4]	-47.7	2.46	52 / 30	EAGLE-3 K2	+65.8 [+59.4, +72.5]	-47.3	1.99	65 / 44
EAGLE-3 K3	+53.8 [+46.5, +62.5]	-48.8	2.20	62 / 41	EAGLE-1 K3	+59.7 [+51.4, +68.9]	-42.2	2.27	63 / 37
N-gram (16)	+8.0 [+2.7, +13.8]	-23.2	1.41	52 / 31	Draft K3	+56.6 [+50.6, +63.2]	-51.3	3.18	58 / 33
EAGLE-1 K5†	-44.1 [-44.4, -43.9]	+49.2	1.01	54 / 30	K5	+54.6 [+45.3, +64.7]	-51.7	4.19	59 / 33
					Draft K7	+48.7 [+37.1, +61.7]	-49.3	4.93	62 / 33
					EAGLE-1 K5	+45.3 [+35.8, +56.3]	-36.4	2.43	58 / 33
					N-gram K5	-7.6 [-10.9, -4.1]	-14.1	1.93	67 / 48

Speed and energy in % relative to no speculation. †Failed the sanity check (accept length ≈ 1 : drafts almost never accepted); excluded.

and ran 44% slower than no speculation; the sanity check excluded it, consistent with earlier observations of this configuration. In an earlier, less controlled benchmark, SGLang’s n-gram method had shown implausible throughput (above 900 tokens/s). Under this protocol, where each prompt is seen once per process, it shows +8% with an accept length of 1.41. This supports, but does not prove, the explanation that n-gram drafting previously benefited from memory of its own earlier answers to repeated identical prompts.

Asynchronous scheduling. Disabling vLLM’s asynchronous scheduling made EAGLE-3 faster, not slower (+79.6% versus +74.0% at $K=3$, all prompts). Disabling it, as vLLM does for the draft model, is therefore not a handicap to K5 at batch size 1. We do not have an explanation for this.

Fidelity in serving engines. In both engines two non-speculative baseline runs produce identical output (100%). Every speculative method, however, diverges from its engine’s baseline at some point in most responses: on average 52–72% of each output matches before the first differing token, and 30–48% of outputs are fully identical (Table II). This holds for EAGLE-3 as much as for K5, and even for SGLang’s EAGLE-1, which accepted almost no drafts (54% mean match). We interpret this as a numerical effect of running the engines in speculative mode in bfloat16: verification processes several tokens per forward pass, and the resulting rounding differences can flip near-tied greedy choices, after which the texts continue differently. Speculative decoding is lossless in exact arithmetic; in practice, in these engines, its output should

TABLE III
CONTROLLED, CACHE-FREE COMPARISON (NO KV-CACHE IN EITHER ARM). N=10 TRIALS PER PROMPT, MEAN $\pm$ S.D.

Prompt	Base tok/s	Spec. tok/s	Speedup	Energy Δ	Accept
Poem	40.16 $\pm$ 0.28	43.55 $\pm$ 0.31	+8.4%	-32.7%	42.5%
Physics	40.05 $\pm$ 0.18	49.64 $\pm$ 0.43	+24.0%	-39.2%	51.7%
Code	40.04 $\pm$ 0.23	73.46 $\pm$ 0.74	+83.5%	-60.8%	85.4%

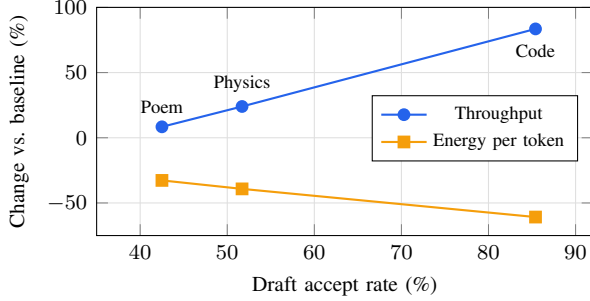


Fig. 2. Cache-free comparison: throughput gain and energy reduction both grow with the draft accept rate, as the mechanism predicts.

be expected to differ from non-speculative output after some point, for every method tested.

IV. CONTROLLED MECHANISM ISOLATION

Separately from the engine comparison, we reimplemented the fixed-window draft-verify loop in plain PyTorch/Transformers, with *no* key-value cache in either arm, so that both arms perform identical full recomputation and only the mechanism differs. Absolute throughput is therefore far below a production server; the relative comparison is the result. Three reference prompts, 10 trials each, closed-loop warmup that converged (241 s), and residual drift checks with $R^2 \leq 0.037$ for every metric. Output matched the non-speculative baseline token for token on every prompt (100% fidelity).

Table III and Figure 2 show the expected behaviour: gains grow with the draft accept rate, from +8% on the least predictable prompt (poetry) to +84% on code. During speculative decoding the GPU draws less power (346–364 W versus 463–466 W) despite holding both models, consistent with fewer full target forward passes per token. Accept rates in this harness (42.5 / 51.7 / 85.4%) agree closely with those measured by vLLM’s own independent draft-model implementation on the same three prompts in an earlier run (43.3 / 57.8 / 85.0%).

A KV-cached version of this plain Python harness did *not* reproduce a speedup: speculation was slower than cached baseline decoding on less predictable prompts. A simple cost model indicates why: in eager Python, a single forward pass of the 1B draft model costs a large fraction of the 8B target’s per-call time (dominated by per-call overhead rather than computation), which removes the draft model’s advantage. The model’s predictions match the measurements, but per-call times were inferred rather than measured directly. The

practical conclusion is that the mechanism needs an optimized engine (as in Section III) to pay off with caching enabled.

V. DISCUSSION AND LIMITATIONS

What the results support. Against a properly configured EAGLE-3, the fixed-window draft model is slower in both engines (8–14%) and comparable on energy per token (within about $\pm 10\%$, in opposite directions in the two engines). It clearly outperforms n-gram drafting and matches EAGLE-1. Its practical relevance is therefore not as a faster alternative to EAGLE-3, but as the option that works where no trained EAGLE head exists (fine-tuned, custom or newly released target models), delivering most of EAGLE-3’s throughput gain and a comparable energy saving using an off-the-shelf small model from the same family.

Remaining asymmetries. In SGLang, tree drafting was evaluated for EAGLE-3 but not for the draft model. In vLLM, the draft model runs on an older model runner than EAGLE-3. vLLM’s EAGLE-3 was evaluated in chain mode only. Any of these could shift the margins; we do not expect them to reverse the speed ordering, but that has not been tested.

Scope. All results are for batch size 1, 250-token responses, one target/draft model pair, and one consumer GPU under WSL2 without clock locking. Speculative decoding’s advantage is known to depend on batch size, and datacenter serving typically batches many requests; the energy results should not be extrapolated to batched serving without measurement. The 27 prompts are original prompts in MT-Bench’s categories, not a standard benchmark set. Two repetitions per configuration were run.

VI. REPRODUCIBILITY

Code and raw telemetry are available at <https://github.com/Creepybits/sdsie-fixed-k5-speculative-decoding> (Apache-2.0). The engine comparison is produced by `benchmarks/likeforlike_suite.py` (orchestration), `likeforlike_worker.py` (one engine and configuration per process) and `likeforlike_analyze.py` (statistics and report); Figure 1 and Tables I–II are taken from `telemetry/likeforlike/20260925_035521_full/` (`report.md`, `summary.json`), which also records the run order seed, per-run engine settings and software versions. Table III is traceable to `telemetry/ablation_results.json`, produced by `benchmarks/benchmark_ablation.py`.

VII. CONCLUSION

In a like-for-like comparison inside vLLM and SGLang, a fixed-window draft-model configuration (1B drafting $K=5$ tokens for 8B) raises throughput by 60–79% and halves GPU energy per token at batch size 1. A well-configured EAGLE-3 is faster still, by 8–14%, with energy per token within about 10% either way. The simple draft model therefore remains a strong, broadly applicable baseline, particularly where no trained EAGLE head is available, rather than a replacement for EAGLE-3. The benchmarking protocol itself, which caught a

configuration that looked favourable in small preliminary runs and was not, may be the most transferable part of this work.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023.
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating large language model decoding with speculative sampling,” arXiv:2302.01318, 2023.
- [3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symp. Operating Systems Principles (SOSP)*, 2023.
- [4] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [5] J. Gante, “Assisted generation: a new direction toward low-latency text generation,” Hugging Face blog, 2023.
- [6] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE: Speculative sampling requires rethinking feature uncertainty,” in *Proc. 41st Int. Conf. Machine Learning (ICML)*, 2024.
- [7] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE-3: Scaling up inference acceleration of large language models via training-time test,” arXiv:2503.01840, 2025.
- [8] S. Agrawal, W. Jeon, and M. Lee, “AdaEDL: Early draft stopping for speculative decoding of large language models via an entropy-based lower bound on token acceptance probability,” arXiv:2410.18351, 2024.
- [9] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.